

Rapport de projet

ChatSystem

FERRERE HOAREAU Anthony
MATHIEU Titouan
4IR A2 - Promotion 60
Année universitaire 2025/2026

Rapport de projet

ChatSystem

FERRERE HOAREAU Anthony

MATHIEU Titouan

4IR A2 - Promotion 60

Année universitaire 2025/2026

INSA Toulouse

135, Avenue de Rangueil

31077 Toulouse Cedex 4 - France

www.insa-toulouse.fr

Table des matières

1	ChatSystem	1
1.1	Description	1
1.2	Fonctionnalité	1
1.3	Arboraisance du projet	1
1.4	Installation, compilation et tests	2
1.5	Tech Stack	2
1.6	Testing Policy	3
1.7	Highlights	3

1 ChatSystem

1.1 Description

Ce projets est une application distribuer où les utilisateur peuvent se connecter avec un nom d'utilisateur, voir la liste de tous les autres utilisateur connecter à l'application avec leur nom d'utilisateur. De plus cette application n'autorise pas plusieurs utilisateur à avoir le même noms d'utilisateur.

1.2 Fonctionnalité

- Analyse du réseau pour voir les autres utilisateur connecter à l'application.
- Connection et déconnection de l'application.
- Changer de nom d'utilisateur

1.3 Arborescence du projet

```
.
  database_explaine.md
  info.txt
  java-report.md
  metadata.yml
  pom.xml
  README.md
  src
    main
      java
        chatsystem2526
          b1
            mathieu
              ferrerehoareau
                chatsystem.java
                controller
                  controller.java
                  PacketHandler.java
                Data_user
                  User.java
                  Userlist.java
                graphic_interface
                  backgroundPanel.java
                  ChangeLogin.java
                  ChatWindow.java
                  ClosableTabComponent.java
                  MainWindow.java
                  WindowLogin.java
                network_tcp
                  database.java
                  localAddressFinder.java
                  Net_tcp_clt.java
                  Net_tcp_srv.java
                network_udp
                  BroadcastAddressFinder.java
                  UDPBroadcaster.java
                  UDPListenerThread.java
```

```

                                UUID_User.txt
resources
    neuron background.jpg
test
    java
        chatsystem2526
            b1
                mathieu
                    ferrerehoareau
                        Controller1Test.java
                        Controller2Test.java
                        Controller3Test.java
                        DatabaseTest.java
                        Data_userTest.java
                        tcp_Test.java

```

1.4 Installation, compilation et tests

- Compilation
 - Avec tests : `mvn clean package`
 - Sans tests : `mvn clean package -DskipTests`
- Test : `mvn test`
- Exécution : `java -jar target/ChatSystem-1.0-SNAPSHOT-jar-with-dependencies.jar`

1.5 Tech Stack

- **Swing (GUI)** : Le choix de Swing pour l'interface graphique a été directement guidé par le programme du cours. En tant que bibliothèque graphique standard de Java, elle a permis de construire une application de bureau portable sans dépendances externes. Comme Swing fonctionne avec un seul thread, il faut faire attention lorsque le programme utilise plusieurs tâches en même temps. Les mises à jour de l'interface doivent donc être faites avec *SwingUtilities.invokeLater*.
- **UDP (Discovery Status)** : Employé pour la découverte des utilisateurs et la diffusion des changements d'état (connexion, déconnexion, changement de pseudo). Sa nature non-connectée est idéale pour le broadcast sur un réseau local, où l'on souhaite notifier tout le monde sans la charge de connexions multiples. La perte occasionnelle d'un paquet de mise à jour est un compromis acceptable pour cette efficacité.
- **TCP (Chat)** : Utilisé pour les sessions de conversation privées entre deux utilisateurs. Il offre un canal de communication fiable et ordonné, garantissant que les messages arrivent dans le bon ordre et sans corruption, ce qui est essentiel pour la cohérence d'un historique de chat. L'implémentation actuelle, où chaque client ouvre un serveur sur le même port fixe (3333).
- **SQLite (Database)** : Utilisé pour stocker l'historique des conversations localement. Étant une base de données légère et sans serveur, elle s'intègre facilement à l'application sous forme d'un simple fichier, sans nécessiter l'installation d'un service externe. C'est une solution robuste et pertinente pour une application de bureau stockant des données locales.

1.6 Testing Policy

La stratégie de test a reposé sur une approche à deux niveaux pour garantir à la fois la robustesse des composants internes et le bon fonctionnement du système dans des conditions réelles.

- **Tests unitaires et d'intégration automatisés (JUnit)** : Des tests ont été écrits pour valider les briques logiques fondamentales. Cela inclut des tests sur les opérations de base de données (*DatabaseTest*), des tests plus complexes simulant des scénarios d'interaction entre plusieurs utilisateurs (*Controller1Test*, *Controller2Test*) dans un environnement contrôlé, et des tests spécifiques comme *tcp-Test.java* qui vérifie de manière robuste la communication TCP client-serveur en utilisant une *BlockingQueue* pour synchroniser le thread du test avec le thread du serveur. Ces tests assurent la fiabilité des composants internes du système.
- **Tests système manuels en réseau** : Le test principal consistait à déployer et exécuter l'application sur plusieurs machines (physiques ou virtuelles) connectées au même réseau local. Cette méthode a permis de valider de manière pragmatique le comportement réel du système : la découverte des utilisateurs via broadcast UDP, l'établissement de sessions de chat en TCP, et la mise à jour en direct des interfaces graphiques, des cas d'usage difficiles à simuler entièrement avec des tests automatisés.

1.7 Highlights

- **Decoupling & Callbacks** -> *network-tcp.Net-tcp-srv's constructor* : La conception du serveur TCP est un bon exemple de découplage. Le constructeur prend en paramètre un *Consumer<String>*, une interface fonctionnelle qui agit comme un "callback". Le serveur TCP se contente de recevoir un message et de le passer à ce *Consumer*, sans connaître la nature du traitement qui sera appliqué. Cela sépare nettement la responsabilité de la couche réseau de la logique applicative.
- **Robust Data Modeling** -> *Data-user.User.java* : La classe *User* utilise un *UUID* pour identifier de manière unique et immuable chaque utilisateur. Le choix d'un identifiant non lié au pseudo (qui peut changer) ou à l'adresse IP (qui peut être volatile) est une décision de conception robuste. Il prévient de nombreuses erreurs potentielles dans un système distribué où l'identification fiable des participants est cruciale.